IEAI-ALP: Intelligent ERP Access Intelligence & Adaptive Least Privilege

Phase-II implementation and measured evaluation

Student: Ansh Goel | Registration: 24BIT0258
Course: BCSE355L Cloud Architecture Design
Report generated: 2026-10-11

1. Abstract

This project implements a reproducible access-governance application with a local web dashboard and an authenticated AWS backend. A published Firewall1 access matrix supports role mining and coverage-preserving structural optimization. A separate Amazon enterprise history supports temporal profiling and observed-outcome offline Q-learning. Human-approved application permissions are versioned, reversible and audited. Source data and recorded outcomes are never replaced by synthetic records or prefilled dashboard metrics.

2. Problem and demonstrated scope

Flat access grants can produce large administrative representations. The goal is to reduce structural policy cost while preserving authorization, explain review signals, and audit permission changes. The Phase-I university ERP scenario supplies the application motivation. Public benchmark labels do not identify actual university departments, students, faculty or ERP actions; those attributes are not fabricated.

The system implements its own authorization layer. A live ERP connector, production threat labels, semantic job-role validation, patent eligibility and formal solver proofs are outside the demonstrated evidence. Instructions and achieved-target assertions in reference documents are not treated as verified implementation facts.

3. Data provenance and engineering

Firewall1 is the variant accompanying the RBAC Maintenance research source. Its raw matrix is immutable, its binary/rectangular structure is checked, and each original 1 becomes one CSV grant. The imported CSV SHA-256 is cf5434adbbb891ba1ae4c9fb172510b81030d28b91ee4aebac95d64524426c91. The measured dataset has 365 users, 709 permissions and 31,919 distinct grants. This variant differs from the commonly cited original's 31,951 grants; no correction is manufactured.

Amazon Access Samples is published by UCI under CC BY 4.0. The unchanged archive supplies actual add/remove access history. 715,651 valid events across 17,607 users and 6,442 target IDs are processed. A further 412 rows have missing/invalid timestamps and are explicitly excluded. The extracted history hash is 83db6197bd4a205b0fd7b0ce931123f16d98f379a26bdcfa2193be363b54bf17. The wide 4.8 GB snapshot is not expanded or falsely included in evaluation. The populations are never joined.

The importer rejects malformed identifiers, records duplicates, preserves opaque IDs, and produces separate derived artifacts. The AWS demo bounds ACL inputs at 25 MiB and 500,000 distinct grants. The larger history is processed offline; its original archive and computed model artifact are stored in private S3.

4. Architecture and AWS services

The local browser authenticates through Cognito and calls API Gateway. Lambda provides metadata, catalogs, review, authorization, model and evidence endpoints. Signed SigV4 URLs upload source CSVs to private encrypted S3. S3 events run the analysis Lambda. Results live in S3; metadata, versions, decisions and audit records live in DynamoDB. CloudFormation/SAM describes infrastructure.

IAM scopes raw/result access and table operations to project resources. IAM Access Analyzer validates actual IAM documents; dataset role exports remain application RBAC labels. CloudWatch records Lambda operations. AWS Budgets sends spending alerts to the configured address.

The frontend remains local at 127.0.0.1:8000. Public registration is disabled; data routes require a JWT. Browser OPTIONS preflight is unauthenticated. Conditional DynamoDB transactions reject stale changes. Local mode uses FastAPI, SQLite WAL and the shared service/analysis modules. The architecture diagram and schema are in ARCHITECTURE.md and API.md.

5. User profiling and risk analysis

ACL profiles contain observed grant count, permission density, rare-grant count and mined memberships. Permissions expose measured user support. The review score is 65 times the rare-grant fraction plus 35 times density. Rarity means support by no more than 5% of known users.

The score is a review cue, not business impact or threat probability. Amazon temporal profiles use actual change counts, additions/removals, target diversity and dates. Attributes absent from a source are not invented.

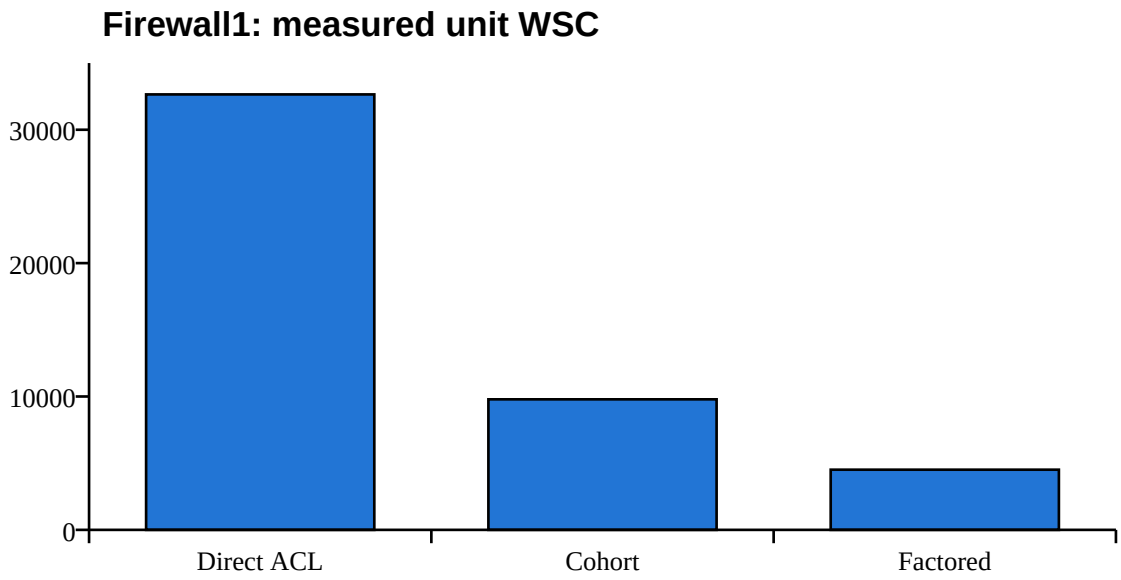
6. Role discovery and optimization

The engine first groups identical access sets. It finds shared-permission support signatures across residual groups and greedily chooses transformations that reduce unit WSC. Search is bounded to 64 transformations. Direct, cohort and factored representations are compared; the lowest measured WSC is selected.

WSC = number of roles + user-role edges + role-permission edges. No role hierarchy or unreported direct grants are introduced. Generated grants are explicitly reconstructed and compared for missing and extra edges. This is an exact coverage check, not a global optimization or SMT proof.

Direct baseline WSC: 32,649.
Cohort baseline: 116 roles; WSC 9,787.
Selected: 132 roles; 1,506 user-role edges; 2,874 role-permission edges; WSC 4,512.
Reduction from direct baseline: 86.1803%.
Measured missing grants: 0; extra grants: 0.

More roles than the cohort baseline can still reduce total structural cost. Role evidence lists actual members and permissions; it does not invent departments.



7. Dynamic authorization and audit

The backend computes the review queue from source evidence. An administrator can retain, approve removal or restore an original grant. Expected-version checks protect concurrent changes. Every effective change stores a parent version, actor and audit event. Original grants/hash remain unchanged.

Authorization uses actual mined role memberships. Live AWS verification confirms allow before removal, deny afterward and allow after restoration. This enforces application authorization, rather than merely changing a displayed number. It does not provision a university ERP account or AWS IAM principal.

8. Reinforcement learning and outcomes

The feedback model learns terminal action values only from genuine administrator reviews; at least five reviews across two users are needed. Validation operations are excluded. A user-disjoint holdout, persisted model and abstention on unseen/ambiguous states are implemented. Permissions always require approval.

The completed historical experiment trains terminal Q-learning on actual Amazon actions and recorded authorization delay. Gamma is zero and alpha is 0.1. States contain only preceding per-user change count/removal fraction and current request-hour bucket. Reward is a normalized negative logarithm of the recorded delay for the observed action. No counterfactual reward or outcome is fabricated. A state/action requires five training observations before recommendation.

Training records: 572,399.
Chronologically later held-out records: 143,252.
Supported evaluation coverage: 99.9979%.
Observed-action agreement: 57.5606%.
Majority-action baseline agreement: 98.4922%.

Whole timestamps remain in one partition. The RL action agreement is lower than the majority baseline. Recorded delay and observational selection prevent claims of improved prediction, causal utility or permission safety. Both results are reported directly; the historical model never changes active policy.

9. Performance measurements

Seven warmed compute runs on Windows-11-10.0.26300-SP0 with Python 3.12.4 give median wall time 113.668 ms and maximum 136.395 ms. A separate Python-allocation run peaks at 8,581,226 bytes, excluding interpreter/RSS.

Sparse sets avoid a dense user-permission matrix. Cached policy artifacts serve repeated requests. Search/pagination avoids full-catalog DOM rendering. System-font fallbacks remove external font requests. No unmeasured large-deployment throughput, sub-millisecond DynamoDB access or formal least-privilege guarantee is claimed.

10. Verification evidence

Nine backend tests pass with real-source grants. Checks cover source hash/counts, duplicate accounting, deterministic reconstruction, metadata, pagination, zero-grant users, empty/invalid inputs, revoke/restore, stale-version rejection, audit and feedback exclusion.

Actual AWS verification covers Cognito authentication, unauthenticated denial, browser preflight, signed S3 upload, Lambda source analysis, DynamoDB versions, removal/restoration authorization and unchanged provenance. IAM validation returns 0 findings for the tested project policy.

Desktop and mobile browser checks cover local and AWS modes, actual dataset metrics, pagination, role evidence and authorization. No mock analytics are used. See evaluation.json, amazon-evaluation.json, aws-verification.json, browser-verification.json and screenshots.

11. Cost and operational controls

The recorded account check found an ACTIVE FREE plan with USD 140.00 remaining credit. Its reported expiry is 2027-03-22T22:59:07.752000+00:00. No account upgrade was performed. This is an observed account state, not a promise that all future usage will be free.

The monthly budget is USD 1 and alerts above USD 0.10 actual spend. It does not stop resources. Sources/results expire after 30 days, DynamoDB uses TTL, and Lambda logs use seven-day retention. No EC2, NAT, RDS, Neptune, paid analyzer or public frontend hosting is deployed. Request-driven services can consume credit, and SAM staging artifacts may persist after teardown.

12. Reproduction and demonstration

Create the virtual environment and install requirements. run_firewall1.py converts/imports the real matrix. prepare_amazon_history.py downloads the official archive and prepares historical evaluation without writing the wide snapshot. evaluate.py reproduces measured optimization. Start-Demo.ps1 launches the local UI. cloud_demo.py configures from stack outputs and verifies AWS; check_browser.py verifies real browser flows.

Demonstrate hashes, inspect roles/permissions, compare WSC, review/restore an actual grant, check authorization, inspect parent versions/audit, show the historical model and baseline, and show AWS resources. Login secrets remain in an ignored private local file.

13. Research and portfolio positioning

The contribution is a reproducible engineering artifact combining exact role reconstruction, measured structural reduction, auditable serverless permission governance and transparent learning evaluation. Role mining, WSC and Q-learning have substantial prior work; implementation alone establishes no patentability or new algorithm.

Future work requires genuine ERP attributes/outcomes, external algorithm baselines, careful causal evaluation and an appropriate prior-art review. No fake department labels or approval outcomes are substituted.

Suggested CV entry: Built an AWS access-governance platform with S3, Lambda, API Gateway, Cognito and DynamoDB; reduced unit structural policy cost by 86.18% on 31,919 real grants with zero reconstruction errors; implemented reversible audited authorization and evaluated offline learning on 715,651 genuine enterprise access changes.

References

Firewall1 source and license: https://onlinerbacfixing.github.io/cybersecurity2019/
Amazon Access Samples: Montanez, K. (2011), UCI, DOI 10.24432/C5JW2K.
https://archive.ics.uci.edu/dataset/216/amazon+access+samples
Benchmark description: https://www3.cs.stonybrook.edu/~liu/papers/RoleMiningData-LPOP20.pdf
DynamoDB transactions:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/transaction-apis.html
AWS Free plans: https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/free-tier-plans.html